

# FrontierAtlas — AI Ecosystem Ingestion Architecture

Technical Specification & System Architecture Submission | GraphOne Demo Task

## 1. Executive Summary & Problem Overview

FrontierAtlas is an asynchronous, fault-tolerant data ingestion pipeline that turns unstructured, noisy web sources into a canonical, provenance-backed intelligence graph spanning Startups, Products, Research Papers, News, and Jobs. Built under strict anti-hallucination rules, every record carries unbroken provenance back to a live, scraped URL.

## 2. System Architecture & Component Interaction

The architecture separates deterministic infrastructure from language understanding. Plain Python code owns crawling, retries, rate-limiting, pre-flight token chunking, freshness verification, fuzzy deduplication, and database persistence. LLM calls are isolated strictly to structuring un-annotated text into Pydantic schemas under prompt injection containment.

| Component        | Implementation                   | Key Responsibilities & Guarantees                                           |
|------------------|----------------------------------|-----------------------------------------------------------------------------|
| Async Crawlers   | httpx.AsyncClient + Semaphore    | Connection pooling, 429/5xx backoff with jitter, 404 rejection              |
| LLM Orchestrator | 3-Tier Fallback Chain            | Gemini 1.5 Flash -> Groq Llama 3 -> DeepSeek; 413 pre-flight chunking       |
| Date & Freshness | DateNormalizer + FreshnessEngine | Normalizes ISO/RFC/relative dates to UTC; strict <=24h filter for News/Jobs |
| Entity Resolver  | RapidFuzz + Seed List (~50)      | Exact -> Alias -> Token-sort fuzzy matching (>=85% confidence threshold)    |
| Deduplicator     | SHA-256 Key Hashing              | Deterministic uniqueness key per entity type; idempotent writes             |
| Storage & Export | SQLAlchemy SQLite + gspread      | Postgres-ready relational schema; 6-tab batched Google Sheets export        |

## 3. Provenance & Zero-Hallucination Integrity Strategy

- **Unbroken Source Provenance:** Every record retains originating ``source_url`` and ``collected_at`` ISO-8601 UTC timestamp.
- **Independent Verification of Dynamic Metrics:** GitHub star counts are fetched directly from GitHub REST API (``/repos/{owner}/{repo}``), NEVER from LLM outputs.
- **Strict Nullability & Rejection:** Missing or ambiguous fields (employee counts, founding dates, prices) default to ``null`` rather than model guesses.
- **Prompt Injection Containment:** Untrusted webpage content is enclosed inside `` ``` tags with instructions rendered inert.

## 4. Scalability Strategy (1,000 -> 10,000 -> 500,000+ Records)

To scale from the demo run to 500,000+ continuous records, the architecture decouples pipeline stages via message queues and stateless worker nodes without altering core extraction logic.

| Scale Tier   | Bottleneck Identified                    | Architectural Scaling Solution                                 |
|--------------|------------------------------------------|----------------------------------------------------------------|
| 1,000 (Demo) | Single-process asyncio loop, SQLite file | In-memory async queue, SQLite with SQLAlchemy ORM              |
| 10,000       | LLM rate limits (429), API concurrency   | Redis Streams queue, provider round-robin, worker pool (N=5)   |
| 100,000      | Database write contention, HTML parsing  | PostgreSQL database, Kafka partitions, Playwright browser pool |
| 500,000+     | LLM costs, deduplication index size      | Local fine-tuned SLM (vLLM/TGI), Redis Bloom filters, PgVector |

## 5. Freshness & Date Normalization Engine

For News and Job records, publication dates are extracted from JSON-LD (``datePublished``), OpenGraph meta tags, HTML ``<time`` elements, or relative text ('2 hours ago'). All dates are parsed into UTC datetimes. Articles/Jobs older than 24 hours or with unparseable dates are strictly rejected, guaranteeing 100% freshness compliance.

## 6. Entity Resolution & Canonicalization Protocol

Entity resolution maps heterogeneous text variants ('OpenAI Inc.', 'Open AI', 'DeepMind Technologies') to a ground-truth seed list of ~50 canonical AI entities using a four-stage pipeline: (1) Normalization (lowercase, legal-suffix stripping), (2) Exact Match, (3) Alias Dictionary Match, (4) RapidFuzz ``token_sort_ratio`` fuzzy matching ( $\geq 85\%$  score). Unresolved entities are left unmerged to prevent false entity coalescing.

## 7. Performance Metrics & Empirical Run Results

The table below details real measured outcomes from pipeline execution against live endpoints:

| Pipeline Entity / Stage | Source Endpoints                                     | Target Count  | Freshness / Verification   | Status    |
|-------------------------|------------------------------------------------------|---------------|----------------------------|-----------|
| Research Papers         | ArXiv Atom REST API                                  | 1,000         | GitHub API Stars Enriched  | COMPLETED |
| AI Startups             | Hugging Face Orgs / GitHub                           | 1,000         | Entity Resolved (~50 Seed) | COMPLETED |
| AI Products             | Hugging Face Spaces & Models                         | 1,000         | Pricing Type Validated     | COMPLETED |
| AI News                 | 5 RSS AI Outlets (TechCrunch, VB, MIT, etc.)         | 24h Monitored | Strict <=24h Verified      | COMPLETED |
| AI Jobs                 | 5 Job Boards (RemoteOK, Remotive, WWR, etc.)         | 24h Monitored | Strict <=24h Verified      | COMPLETED |
| Google Sheets Export    | 6 Tabs (Startups, Products, Papers, Jobs, News, abs) | 6 Tabs        | Batched Writes             | COMPLETED |

## 8. Deliverables & Audit Checklist

- **Python Codebase** (``src/``): Modular, production-ready package with test suite (``tests/``).
- **Relational Database** (``frontier_atlas.db``): SQLite database matching Pydantic schemas with unique dedup keys.
- **Google Sheets Dashboard**: 6-tab synchronized public export with Entity Mapping Log.
- **Architecture Document** (``architecture.pdf``): Concise 3-page submission document detailing engineering trade-offs.